

# Programmering

Abstrakte klasser  
og indkapsling

# Indkapsling 1

- **private** metoder og attributter er **kun** tilgængelig i klassen selv
- Hvis der ikke står noget er metoder og attributter synlig i samme package. Kaldet package private
- **protected** metoder og attributter er synlig i samme package og i alle subklasser.
- **public** metoder og attributter kan ses af alle.

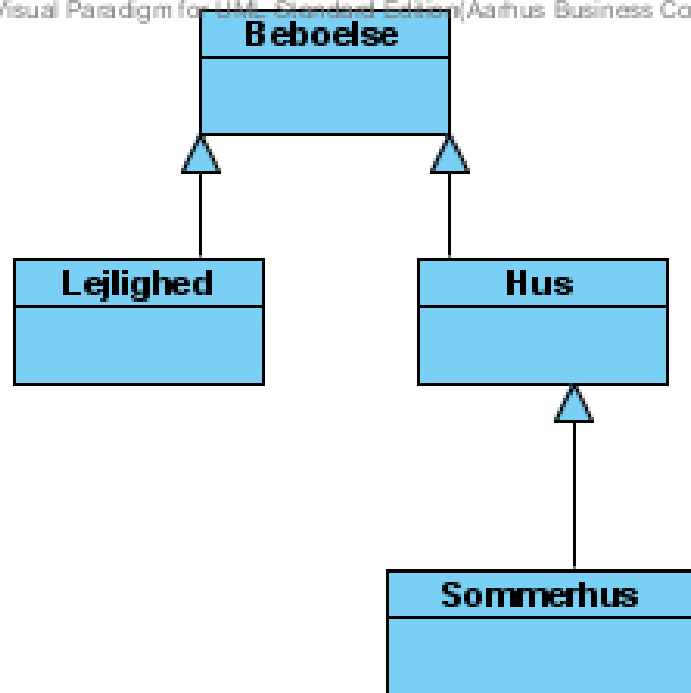
# Indkapsling 2

- En **attribut** skal altid være private, men dens get og set **metode** kan være public eller protected – dvs. attributtens implementation er indkapslet
- Det betyder at vi ved at attributten kun ændres ved kald til de public/protected metoder.
- Gør det lettere at teste og giver mere vedligeholdelsesvenlige programmer

# Abstrakt klasse

## Beboelses klassen fra sidst

Visual Paradigm for UML - Standard Edition / Aarhus Business College



```

public class Beboelse {
    private String adresse;
    private int kvm;

    public Beboelse(String adresse, int kvm) {
        this.adresse = adresse;
        this.kvm = kvm;
    }

    public String getAdresse() {
        return adresse;
    }

    public int getKvm() {
        return kvm;
    }
}
  
```

- Vil I lave objekter af klassen Beboelse ?
- Har klassen Beboelse et formål ?

# Abstrakt klasse

- En abstrakt klasse er en klasse, der ikke kan laves objekter af
- En abstrakt klasse indeholder **fælles implementation og evt. fælles "interface"**
- En klasse skal være abstrakt når
  - den modellerer en abstraktion med implementation
  - der er metoder uden implementation

# Abstrakt klasse

```
public abstract class Beboelse {
    private String adresse;
    private int kvm;

    public Beboelse(String adresse, int kvm) {
        this.adresse = adresse;
        this.kvm = kvm;
    }

    public String getAdresse() {
        return adresse;
    }

    public int getKvm() {
        return kvm;
    }

    public void setKvm(int kvm) {
        this.kvm = kvm;
    }

    public abstract int ejVaerdi() ;
}
```

Alle beboelser skal have  
en ejVærdi() – men de  
konkrete klasser  
bestemmer hvordan den  
skal beregnes

Ingen kode blok kun ;

# Abstrakt klasse (fortsat)

- En abstrakt klasse angives med nøgleordet **abstract**
- Kan indeholde attributter og constructor samt konkrete og abstrakte metoder
- Behøver ikke implementere arvede abstrakte metoder
- En klasse med abstrakte metoder skal være abstrakt

Hvordan kaldes en constructor i en abstrakt klasse ?

*Den kaldes via **super** fra de konkrete subklasser*

# Konkret klasse

- En konkret klasse er en klasse, der kan laves objekter af
- En klasse er konkret når den ikke er abstrakt
- Kan indeholde attributter og konkrete metoder
- Kan ikke indeholde abstrakte metoder
- Arvede abstrakte metoder skal være implementeret

# Abstrakt metode

- En abstrakt metode er en metode uden implementation – er kun et interface
- En abstrakt metode angives med nøgleordet `abstract` og afsluttes med et semikolon i stedet for en kodeblok
- `public abstract void abstractMetode();`

# Polymorfi (fortsat)

- En references erklærede type er den referende i programteksten (statisk type)
- En references aktuelle type er typen af det refererede objekt (dynamisk type)
- `X x = new Y(); x.metode();`
  - X er den erklærede type for x (statiske type)
  - Y er den aktuelle type for x (dynamiske type)
- Den erklærede type afgør **hvad** der kan gøres
- Den aktuelle type afgør **hvordan** det gøres

# Polymorfi (også på abstrakte klasser)

```
public static void main(String[] args) {  
  
    Beboelse[] beboelser = new Beboelse[5];  
    beboelser[0] = new Hus("Vandet 2", 110, 1500);  
    beboelser[1] = new Sommerhus("Vandet 4", 66, 1000, false);  
    beboelser[2] = new Hus("Havet 1", 187, 2000);  
    beboelser[3] = new Sommerhus("Havet 2", 98, 900, true);  
    beboelser[4] = new Hus("Havet 3", 100, 600);  
  
    int sum = 0;  
    for (Beboelse beboelse: beboelser) {  
        sum = sum + beboelse.ejVaerdi();  
    }  
}
```

# final

- Angives med nøgleordet **final**
- En final metode kan ikke overrides
- En final klasse kan ikke specialiseres
- Sikrer mod uønsket modifikation af kode ved hjælp af override